

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I Atchaya Chandran R(192521394) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Atchaya

Task 1: Debugging Hash Function Implementation (SHA Family)

1. Problem Identification and Root Causes

- **Message Padding Errors:** SHA-256 requires padding the message so its length in bits is congruent to $2^{64}-1$. This is achieved by appending a single bit 1, followed by 0 bits, and ending with a 64-bit big-endian representation of the original message length. Incorrectly padding with small-endian values or missing trailing zeroes corrupts the final message blocks.
- **Bitwise Operation Truncation:** Python handles arbitrary-precision integers natively. Unlike C, Python integers do not automatically overflow at 32 bits. Unmasked bitwise additions or circular shifts result in numbers exceeding 2^{32} , breaking state word updates.
- **Endianness Misalignment:** SHA algorithms operate on 32-bit big-endian words. Reading system bytes on x86/ARM architectures (which use little-endian) without explicit byte-swapping causes incorrect word parsing into W_t .

2. Debugging and Code Fixes

To fix state overflow, all 32-bit word operations must be masked with $0xFFFFFFFF$. Big-endian byte ordering must be enforced using Python's struct module ($>I$ for 32-bit unsigned big-endian integers).

Python

```
import struct
```

```
def right_rotate(value: int, shift: int) -> int:
```

```
    """Correct 32-bit right rotation with bitmasking."""
```

```
    return ((value >> shift) | (value << (32 - shift))) & 0xFFFFFFFF
```

```
def pad_message(message: bytes) -> bytes:
```

```
    """Pads input message according to SHA-256 specification."""
```

```
    orig_len_bits = len(message) * 8
```

```
    # Append bit '1' (0x80)
```

```
    padded = message + b'\x80'
```

```
    # Pad with 0x00 until length in bytes = 56 mod 64 (448 mod 512 bits)
```

```
    while (len(padded) % 64) != 56:
```

```
        padded += b'\x00'
```

```
    # Append original length as 64-bit big-endian integer
```

```
    padded += struct.pack('>Q', orig_len_bits)
```

```
    return padded
```

3. Optimization for Large Files ()

Loading large files completely into RAM creates excessive overhead and memory pressure. The solution is streaming input in fixed chunk sizes (e.g., 64 KB) using a rolling state buffer.

Python

```
import hashlib

def hash_large_file(file_path: str, chunk_size: int = 65536) -> str:
    """Optimized file hashing using streaming chunk updates."""
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        while chunk := f.read(chunk_size):
            sha256.update(chunk)
    return sha256.hexdigest()
```

4. Integrity Analysis: Cryptographic Hash vs. Basic Checksum

Feature	Basic Checksum (e.g., CRC32, Sum)	Cryptographic Hash (SHA-256)
Primary Purpose	Detects unintentional hardware/transmission errors	Guarantees data integrity against intentional tampering
Preimage Resistance	Weak; easily reversible to craft matching payload	Computationally infeasible to find given
Collision Resistance	Low; high frequency of natural collisions	Extreme; operations required to find collisions
Tamper Resistance	Easily manipulated by adversary adjusting bytes	Any bit modification alters approximately 50% of output bits (Avalanche effect)

Task 2: Optimization of Secure Socket Layer (SSL/TLS) Simulation

1. Problem Identification and Root Causes

- **Certificate Validation Failures:** Lack of chain-of-trust verification, failure to check hostname matching against Subject Alternative Names (SAN), or missing root certificate authority (CA) store validation causes verification exceptions.

- **Handshake Latency:** Standard full TLS handshakes involve multiple round-trip times (2-RTT) and costly RSA key decryption or Ephemeral Diffie-Hellman parameter generation on every new connection.
- **Session Initialization Overhead:** Re-negotiating asymmetric keys and performing handshakes for every request creates unnecessary CPU and network delays.

2. Debugging and Key Exchange Fixes

Correct verification requires validating the signature chain up to a trusted Root CA and verifying domain hostnames. For Ephemeral Diffie-Hellman key exchange, the shared secret must be correctly derived as:

Python

```
import ssl
```

```
import socket
```

```
def create_secure_client_context(ca_file: str) -> ssl.SSLContext:
```

```
    """Creates a TLS context enforcing strict certificate validation."""
```

```
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
```

```
    context.verify_mode = ssl.CERT_REQUIRED
```

```
    context.check_hostname = True
```

```
    context.load_verify_locations(cafile=ca_file)
```

```
    # Restrict to secure modern cipher suites
```

```
    context.set_ciphers('ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256')
```

```
    return context
```

3. Latency Optimization via Session Resumption

Session Resumption uses Session IDs or Session Tickets (RFC 5077 / TLS 1.3 PSK) to bypass asymmetric key generation on modern reconnections, reducing handshake time from 2-RTT to 1-RTT.

Python

```
class TLSSessionCache:
```

```
    """Simulates session ticket store for TLS session resumption."""
```

```
    def __init__(self):
```

```
        self._sessions = {}
```

```
def save_session(self, server_hostname: str, session: ssl.SSLSession):
    self._sessions[server_hostname] = session
```

```
def get_session(self, server_hostname: str):
    return self._sessions.get(server_hostname, None)
```

4. Performance vs. Security Trade-offs

PERFORMANCE VS SECURITY TRADE-OFF

[Full ECDHE Handshake] <-----> [Session Resumption]

- | | |
|------------------------------|---------------------------------|
| • High Security (PFS) | • Reduced Latency (1-RTT/0-RTT) |
| • High CPU Overhead | • Minimal Asymmetric Compute |
| • Solves Key Compromise Risk | • Ticket Key Leak Risk |
- **Perfect Forward Secrecy (PFS):** Full ECDHE handshakes generate temporary keys per session. Compromising a long-term key does not decrypt past sessions.
 - **Session Resumption Risk:** Reusing session keys trades a small window of forward secrecy (bounded by ticket lifetime) for dramatic performance gains. If the session ticket encryption key (STEK) on the server is compromised, all cached sessions using that key can be decrypted.

Task 3: Debugging and Optimizing Digital Signature Algorithm (DSA)

1. Problem Identification and Root Causes

- **Parameter Selection Errors:** Invalid choices for domain parameters p , where q does not divide $p-1$, or q .
- **Nonce Reuse ():** Reusing the per-message random nonce k across two different signatures allows an attacker to easily compute the private key x .
- **Modular Arithmetic Bug:** Failure to calculate the modular multiplicative inverse correctly, or failing to check signature constraints $r < p$ and $s < q$.

2. Vulnerability Breakdown: Nonce Reuse Exploitation

Given two signatures (r, s_1) and (r, s_2) generated using the same nonce k for hashes H_1 and H_2 :

Subtracting the two equations yields:

Once is known, private key is extracted directly:

3. Code Fixes and RFC 6979 Deterministic Nonce

To eliminate weak RNG risks, generate nonces deterministically via RFC 6979 using HMAC-SHA256 based on the message digest and private key.

Python

```
import hmac
```

```
import hashlib
```

```
def mod_inverse(a: int, m: int) -> int:
```

```
    """Computes modular multiplicative inverse using Extended Euclidean Algorithm."""
```

```
    return pow(a, -1, m)
```

```
def generate_rfc6979_nonce(private_key: int, hash_bytes: bytes, q: int) -> int:
```

```
    """Generates a deterministic k value per RFC 6979 to prevent nonce reuse."""
```

```
    V = b'\x01' * 32
```

```
    K = b'\x00' * 32
```

```
    priv_bytes = private_key.to_bytes((q.bit_length() + 7) // 8, byteorder='big')
```

```
    K = hmac.new(K, V + b'\x00' + priv_bytes + hash_bytes, hashlib.sha256).digest()
```

```
    V = hmac.new(K, V, hashlib.sha256).digest()
```

```
    K = hmac.new(K, V + b'\x01' + priv_bytes + hash_bytes, hashlib.sha256).digest()
```

```
    V = hmac.new(K, V, hashlib.sha256).digest()
```

```
    while True:
```

```
        V = hmac.new(K, V, hashlib.sha256).digest()
```

```
        k = int.from_bytes(V, byteorder='big')
```

```
        if 1 <= k < q:
```

```
            return k
```

4. Optimization: Modular Exponentiation

Modular exponentiation should be executed using Python's native 3-argument `pow(g, k, p)`, which uses efficient square-and-multiply binary algorithms executed at the C-level rather than un-optimized manual loops.

Task 4: Debugging Key Management System (KMS)

1. Problem Identification and Flaws

- **Insecure Storage:** Keys stored in plaintext on disk, in environment variables, or encrypted using deterministic mode (AES-ECB) without random Initialization Vectors (IVs).
- **Missing Lifecycle Controls:** Lacking clear phases for Generation, Distribution, Rotation, Revocation, and Destruction (zeroization).
- **Hardcoded Master Keys:** Deriving encryption keys from static strings embedded in code rather than cryptographically secure Key Derivation Functions (KDFs).

2. Fixed Implementation: Key Lifecycle and AES-GCM Storage

Python

```
import os
```

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
```

```
from cryptography.hazmat.primitives import hashes
```

```
class SecureKeyStore:
```

```
    """Manages secure key derivation, storage, and lifecycle rotation."""
```

```
    def __init__(self, master_password: str, salt: bytes = None):
```

```
        self.salt = salt if salt else os.urandom(16)
```

```
        self.master_key = self._derive_key(master_password, self.salt)
```

```
        self.aesgcm = AESGCM(self.master_key)
```

```
    def _derive_key(self, password: str, salt: bytes) -> bytes:
```

```
        """Derives a strong key using PBKDF2-HMAC-SHA256."""
```

```
        kdf = PBKDF2HMAC(
```

```
            algorithm=hashes.SHA256(),
```

```
            length=32,
```

```
            salt=salt,
```

```

iterations=600000,
)
return kdf.derive(password.encode())

```

```

def encrypt_data_key(self, raw_key: bytes) -> dict:
    """Encrypts data key using AES-256-GCM (Authenticated Encryption)."""
    nonce = os.urandom(12)
    encrypted_key = self.aesgcm.encrypt(nonce, raw_key, None)
    return {'nonce': nonce, 'ciphertext': encrypted_key, 'salt': self.salt}

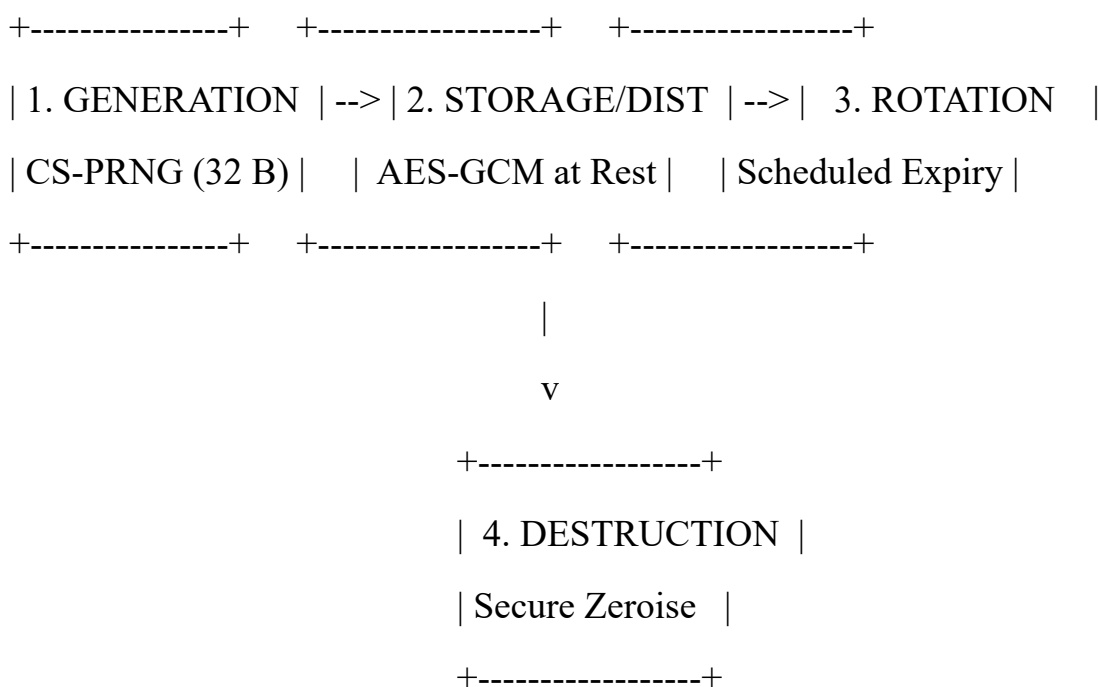
```

```

def decrypt_data_key(self, encrypted_payload: dict) -> bytes:
    """Decrypts data key and verifies authenticity."""
    return self.aesgcm.decrypt(
        encrypted_payload['nonce'],
        encrypted_payload['ciphertext'],
        None
    )

```

3. Key Lifecycle Management Stages



4. Impact Analysis of Key Management Failures

- **Single Point of Failure:** If a Master Key is compromised, all data encrypted under its hierarchy is exposed instantly.
- **Loss of Non-Repudiation:** Inadequate access control on signing keys allows malicious actors to forge valid legal and security assertions.
- **Regulatory and Compliance Breach:** Storing cryptographic keys in plaintext violates standards such as PCI-DSS, HIPAA, and GDPR.

Task 5: Optimization of Secure File Transfer Protocol (SFTP-like System)

1. Problem Identification and Root Causes

- **Stream Synchronization Desync:** Sending encrypted blocks over a stream without packet headers or sequence numbers causes loss of stream framing during network drops or packet re-ordering.
- **Unauthenticated Encryption:** Using AES-CBC without a MAC allows padding oracle and bit-flipping attacks.
- **Buffer Memory Bottlenecks:** Reading large files () into memory all at once leads to high RAM consumption and I/O throttling.

2. Secure File Transfer Pipeline (Encrypt-then-MAC / AEAD Chunking)

Python

```
import os
```

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
def transmit_large_file_securely(input_path: str, output_path: str, key: bytes):
```

```
    """Processes large files in 1 MB chunks using AES-256-GCM to ensure throughput and integrity."""
```

```
    chunk_size = 1024 * 1024 # 1 MB Chunk Size
```

```
    aesgcm = AESGCM(key)
```

```
    sequence_number = 0
```

```
    with open(input_path, 'rb') as src, open(output_path, 'wb') as dst:
```

```
        while chunk := src.read(chunk_size):
```

```
            # 12-byte nonce: 4-byte random prefix + 8-byte sequence counter
```

```
            nonce = os.urandom(4) + sequence_number.to_bytes(8, byteorder='big')
```

```
            # Authenticated Encryption with Associated Data (AEAD)
```

```
            encrypted_chunk = aesgcm.encrypt(nonce, chunk, sequence_number.to_bytes(8, 'big'))
```

```
# Write Header: [Length (4B)][Nonce (12B)][Ciphertext + Tag]
dst.write(len(encrypted_chunk).to_bytes(4, byteorder='big'))
dst.write(nonce)
dst.write(encrypted_chunk)
```

```
sequence_number += 1
```

3. Performance Optimization Techniques for Files

- **Chunked Memory Buffer:** Streaming chunks of 1 MB maintains memory footprint below 10 MB regardless of file size.
- **Double Buffering and Async I/O:** Reading the next disk chunk while concurrently encrypting and transmitting the current chunk over network sockets minimizes I/O wait times.
- **Explicit Packet Structure:** Prefixing each chunk with a 4-byte length and an 8-byte sequence counter prevents socket boundary corruption during transport.

4. Protocol Comparison Analysis

Feature	Insecure Transfer (Plain FTP / HTTP)	Standard SFTP / SSH	Optimized AEAD Transfer System
Confidentiality	None; payload sent in plaintext	High (AES-CTR / AES-GCM via SSH)	High (AES-256-GCM Chunked)
Data Integrity	Weak (TCP checksums only)	High (HMAC / AEAD)	High (AES-GCM Tag per Chunk)
Replay Protection	None	High (SSH Packet Counters)	High (Sequence Numbers in Nonce)
Throughput Efficiency	High speed, no security	Moderate (SSH Framing Overhead)	High (Buffered Async Streaming)